We use the `prep_data2delta` function to load and prepare our training and validation data sets (*Figure 3.30*). Notice that the function will save a Delta table if the `write2delta` flag is "True" and will return a DataFrame if the value for the `returnDF` flag is "True." Next, in the notebook, we call `prep_data2delta` for the training and validation sets.

Let's talk about data loaders

We load data in a fixed-size batch when fine-tuning or training our deep learning models. Each framework natively supports specific data types; some expand their native formats to other open-source formats. Data scientists sometimes prefer to keep their data (images, in our case) in blob storage and read it directly from storage rather than using Delta tables because they think this avoids additional work. However, we recommend storing images in a Delta table unless you are working with images larger than one gigabyte per image. Storing your images in a Delta table allows you to take advantage of Delta and Unity Catalog's additional benefits, such as lineage of data and models, data version control, duplicate data checks, and quality assurance.

As of today, you have a few options to read your data while working with a PyTorch or PyTorch Lightning framework:

- DeltaTorchLoader (recommended)
- Petastorm
- Reading images directly from Blob/Disk/Volumes

Our recommendation is to use DeltaTorchLoader. It handles data batching, sampling, and multiprocessing while training PyTorch pipelines without requiring a temporary copy of files like with Petastorm. See *Further reading* for more information.

As of now, DeltaTorchLoader, which we are going to use to load and transform our data from Delta into the PyTorch/Lightning dataloader framework for training our model in *Chapter 6*, requires you to have tables in the unmanaged Delta format in Volumes when using UC. Don't worry, the lineage is associated with the same path to the Volume as your main dataset. We'll talk more about lineage in *Chapter 6*. This requirement is due to UC's read/write security permissions with blob storage. The blob storage maintainers do not support these security settings yet. If you are not using UC, you should be able to read Delta tables directly from the managed tables.

> **Note**
>
> Petastorm requires a deeper understanding of certain pitfalls. The most common are memory usage issues due to data caching and the fact that it's using Apache parquet files rather than Delta files, so it consumes all versions of your parquet files.